

LLD Practice Notes for Placements

This file is a placement-focused Low Level Design notebook. For each problem, practice explaining:

- Functional requirements
- Non-functional requirements
- Main entities and relationships
- Class responsibilities
- Important APIs
- Design patterns used
- Edge cases
- Common interviewer follow-ups

General LLD interview flow:

1. Clarify scope and assumptions.
2. List core use cases.
3. Identify entities and relationships.
4. Define class responsibilities.
5. Sketch important methods and data structures.
6. Discuss extensibility, concurrency, validation, and error handling.
7. Walk through one or two flows end to end.

Common principles:

- Single Responsibility Principle: one class should have one clear reason to change.
- Open/Closed Principle: add new behavior through interfaces or strategies where useful.
- Dependency Inversion: high-level services should depend on abstractions, not concrete details.
- Composition over inheritance unless there is a true "is-a" relationship.
- Keep domain objects separate from services that coordinate workflows.
- Do not over-design every problem. Add extensibility where the interviewer asks or where change is obvious.

1. Parking Lot

Problem

Design a parking lot where vehicles can enter, get a ticket, park in a suitable spot, and pay on exit.

Requirements

Functional:

- Support multiple vehicle types: bike, car, truck.
- Support multiple parking spot types: compact, large, bike, handicapped.
- Assign an available compatible spot when a vehicle enters.
- Generate a parking ticket.

- Calculate fee on exit.
- Mark spot as free after exit.
- Support multiple floors and gates.

Non-functional:

- Fast spot lookup.
- Avoid assigning the same spot to two vehicles.
- Easy to add new vehicle types, spot types, and pricing rules.
- Thread-safe entry/exit handling if multiple gates operate at once.

Main Entities

- ParkingLot
- ParkingFloor
- ParkingSpot
- Vehicle
- ParkingTicket
- EntryGate
- ExitGate
- SpotAssignmentStrategy
- PricingStrategy
- Payment

Possible Class Design

```
enum VehicleType {
    BIKE, CAR, TRUCK
}

enum SpotType {
    BIKE, COMPACT, LARGE, HANDICAPPED
}

enum TicketStatus {
    ACTIVE, PAID, LOST
}

abstract class Vehicle {
    private final String licensePlate;
    private final VehicleType type;
}

class ParkingSpot {
    private final String id;
    private final SpotType type;
    private boolean occupied;
    private Vehicle parkedVehicle;

    boolean canFit(Vehicle vehicle) { }
```

```

    void park(Vehicle vehicle) { }
    void unpark() { }
}

class ParkingTicket {
    private final String ticketId;
    private final Vehicle vehicle;
    private final ParkingSpot spot;
    private final long entryTime;
    private Long exitTime;
    private TicketStatus status;
}

interface SpotAssignmentStrategy {
    ParkingSpot findSpot(Vehicle vehicle, List<ParkingFloor> floors);
}

interface PricingStrategy {
    Money calculateFee(ParkingTicket ticket);
}

class ParkingLot {
    private List<ParkingFloor> floors;
    private SpotAssignmentStrategy assignmentStrategy;
    private PricingStrategy pricingStrategy;

    ParkingTicket parkVehicle(Vehicle vehicle) { }
    Money unparkVehicle(String ticketId) { }
}

```

Important Design Choices

- Use Strategy pattern for spot assignment:
 - nearest spot
 - lowest floor first
 - random allocation
 - reserved/handicapped priority
- Use Strategy pattern for pricing:
 - hourly pricing
 - slab pricing
 - vehicle-based pricing
 - weekend pricing
- Use a repository or map for active tickets:
 - `Map<String, ParkingTicket> activeTickets`
- Use locks or synchronized methods around spot allocation and release.

Spot Compatibility Example

- Bike can park in bike, compact, or large spot.
- Car can park in compact or large spot.

- Truck can park only in large spot.

Important APIs

```
ParkingTicket parkVehicle(Vehicle vehicle);  
Invoice exitVehicle(String ticketId, PaymentMode mode);  
List<ParkingSpot> getAvailableSpots(SpotType spotType);  
boolean isFull(VehicleType vehicleType);
```

Edge Cases

- Parking lot is full.
- Ticket is invalid.
- Ticket already paid.
- Payment fails.
- Vehicle tries to exit from a different gate.
- Lost ticket.
- Two gates try to assign the same spot.
- Vehicle type has no compatible spot.

Follow-ups

- How will you support reservations?
 - How will you show available spot count per floor?
 - How will you support EV charging spots?
 - How will you handle dynamic pricing?
 - How will you persist ticket history?
-

2. Elevator System

Problem

Design an elevator system for a building with multiple elevators and floors.

Requirements

Functional:

- Users can request elevator from a floor.
- Users can select destination floor inside elevator.
- Elevator moves up/down and opens/closes doors.
- Multiple elevators should be scheduled efficiently.
- Show current floor and direction.

Non-functional:

- Minimize waiting time.
- Handle concurrent requests.

- Easy to change scheduling algorithm.
- Safe state transitions.

Main Entities

- ElevatorSystem
- ElevatorCar
- Floor
- Button
- Request
- Dispatcher
- ElevatorController
- SchedulingStrategy
- Door

Class Design

```
enum Direction {
    UP, DOWN, IDLE
}

enum ElevatorState {
    MOVING, IDLE, DOOR_OPEN, MAINTENANCE
}

class Request {
    private final int sourceFloor;
    private final int destinationFloor;
    private final Direction direction;
}

class ElevatorCar {
    private final int id;
    private int currentFloor;
    private Direction direction;
    private ElevatorState state;
    private TreeSet<Integer> upStops;
    private TreeSet<Integer> downStops;
    private Door door;

    void addDestination(int floor) { }
    void moveOneStep() { }
    void openDoor() { }
    void closeDoor() { }
}

interface SchedulingStrategy {
    ElevatorCar selectElevator(Request request, List<ElevatorCar> elevators);
}

class ElevatorSystem {
```

```
private List<ElevatorCar> elevators;  
private SchedulingStrategy schedulingStrategy;  
  
void requestElevator(int floor, Direction direction) { }  
void selectFloor(int elevatorId, int destinationFloor) { }  
}
```

Scheduling Strategies

- Nearest elevator:
 - choose closest idle elevator.
 - simple but may be inefficient.
- Direction-aware:
 - choose elevator already moving toward the source floor.
 - better for real systems.
- SCAN algorithm:
 - elevator keeps moving in one direction serving requests, then reverses.
 - similar to disk scheduling.

State Machine

```
IDLE -> MOVING -> DOOR_OPEN -> IDLE  
IDLE -> MAINTENANCE  
MOVING -> MAINTENANCE only after emergency stop
```

Data Structures

- `TreeSet<Integer> upStops`: floors above current floor in ascending order.
- `TreeSet<Integer> downStops`: floors below current floor in descending order.
- Priority queues can also be used.

Edge Cases

- Request from top floor to go up.
- Request from ground floor to go down.
- Elevator overloaded.
- Door blocked.
- Elevator under maintenance.
- Emergency stop.
- Duplicate floor requests.
- Multiple requests at same floor.

Follow-ups

- How do you handle VIP elevators?
- How do you optimize for peak office hours?
- How do you support service elevators?

- How do you simulate elevator movement?
 - How do you handle power failure?
-

3. Splitwise

Problem

Design an expense sharing application where users can create groups, add expenses, split them, and settle balances.

Requirements

Functional:

- Users can add expenses.
- Expenses can be split equally, exactly, or by percentage.
- Users can belong to groups.
- App should show balances.
- Users can settle debts.
- Simplify debts if required.

Non-functional:

- Correct financial calculation.
- Avoid floating point errors.
- Easy to add new split types.
- Maintain transaction history.

Main Entities

- User
- Group
- Expense
- Split
- BalanceSheet
- Settlement
- SplitStrategy
- ExpenseService

Class Design

```
class User {  
    private final String id;  
    private String name;  
    private String email;  
}  
  
class Group {  
    private final String id;
```

```

    private String name;
    private List<User> members;
    private List<Expense> expenses;
}

class Expense {
    private final String id;
    private User paidBy;
    private Money amount;
    private String description;
    private List<Split> splits;
}

class Split {
    private User user;
    private Money amount;
}

interface SplitStrategy {
    List<Split> split(Money amount, List<User> users, SplitMetadata metadata);
}

class ExpenseService {
    void addExpense(String groupId, User paidBy, Money amount, SplitStrategy
strategy) { }
    Map<User, Money> getBalances(String groupId) { }
    List<Transaction> simplifyDebts(String groupId) { }
}

```

Balance Logic

For each expense:

- Payer gets credited by total amount paid.
- Each participant gets debited by their split amount.

Example:

- A pays 900 for A, B, C equally.
- A paid: +900
- A owes: -300
- B owes: -300
- C owes: -300
- Net:
 - A: +600
 - B: -300
 - C: -300

Simplify Debt Algorithm

Use two heaps:

- Max heap for creditors.
- Max heap for debtors by absolute value.

Repeatedly match largest debtor with largest creditor:

```
debtor pays creditor min(debt, credit)
update remaining balances
```

Important APIs

```
void createGroup(String name, List<User> members);
void addExpense(String groupId, String paidByUserId, Money amount, SplitType
splitType);
Map<User, Money> getUserBalances(String userId);
List<Transaction> settleGroup(String groupId);
```

Design Patterns

- Strategy pattern for split logic.
- Factory pattern to create split strategy based on split type.
- Repository pattern for storing users, groups, expenses.

Edge Cases

- Percent splits do not total 100%.
- Exact splits do not total expense amount.
- Negative or zero expense.
- User not part of group.
- Currency mismatch.
- Rounding errors.
- Duplicate settlement.

Follow-ups

- How do you support multiple currencies?
- How do you store audit history?
- How do you handle recurring expenses?
- How do you notify users?
- How do you support partial settlements?

4. Library Management System

Problem

Design a system for managing books, members, borrowing, returning, searching, and fines.

Requirements

Functional:

- Add, remove, and search books.
- A book can have multiple physical copies.
- Members can borrow and return copies.
- Track due dates and fines.
- Reserve books if unavailable.

Non-functional:

- Accurate inventory tracking.
- Fast search by title, author, subject, ISBN.
- Easy to add new fine rules.

Main Entities

- Book
- BookItem
- Author
- Member
- Librarian
- Loan
- Reservation
- Catalog
- FineCalculator

Book vs BookItem

- **Book**: logical book metadata.
 - ISBN
 - title
 - authors
 - publisher
- **BookItem**: physical copy.
 - barcode
 - rack location
 - status

Class Design

```
enum BookStatus {  
    AVAILABLE, LOANED, RESERVED, LOST  
}  
  
class Book {  
    private String isbn;  
    private String title;
```

```
    private List<Author> authors;
}

class BookItem {
    private String barcode;
    private Book book;
    private BookStatus status;
    private String rackLocation;
}

class Member {
    private String id;
    private String name;
    private List<Loan> activeLoans;
}

class Loan {
    private BookItem item;
    private Member member;
    private LocalDate issueDate;
    private LocalDate dueDate;
    private LocalDate returnDate;
}

interface FineCalculator {
    Money calculateFine(Loan loan);
}

class LibraryService {
    Loan issueBook(String memberId, String barcode) { }
    Money returnBook(String barcode) { }
    List<Book> searchByTitle(String title) { }
}
```

Important Rules

- Member cannot borrow more than max allowed books.
- Member cannot borrow if unpaid fine exceeds threshold.
- Reserved copy should not be issued to another member.
- Due date is calculated from issue date and member type.

Edge Cases

- Book copy already loaned.
- Member limit reached.
- Invalid barcode.
- Book returned late.
- Book lost or damaged.
- Reservation expires.
- Same book has multiple copies.

Follow-ups

- How do you support digital books?
 - How do you support multiple library branches?
 - How do you build search indexes?
 - How do you notify users before due date?
 - How do you handle librarian permissions?
-

5. Movie Ticket Booking

Problem

Design a movie booking system like BookMyShow.

Requirements

Functional:

- Search movies by city, theater, language, genre.
- View shows and available seats.
- Select seats.
- Temporarily lock seats.
- Make payment.
- Confirm booking.
- Cancel booking.

Non-functional:

- Prevent double booking.
- Seat selection should be fast.
- Handle payment failure gracefully.
- Support high concurrency.

Main Entities

- Movie
- Theater
- Screen
- Seat
- Show
- ShowSeat
- Booking
- Payment
- SeatLock
- BookingService
- SeatLockProvider

Seat vs ShowSeat

- **Seat**: physical seat in a screen.
- **ShowSeat**: seat availability for a specific show.

This separation is important because the same physical seat is available for different shows.

Class Design

```
enum SeatType {
    NORMAL, PREMIUM, RECLINER
}

enum SeatStatus {
    AVAILABLE, LOCKED, BOOKED
}

class Seat {
    private String id;
    private String row;
    private int number;
    private SeatType type;
}

class Show {
    private String id;
    private Movie movie;
    private Screen screen;
    private LocalDateTime startTime;
    private LocalDateTime endTime;
}

class ShowSeat {
    private Show show;
    private Seat seat;
    private SeatStatus status;
}

class Booking {
    private String id;
    private User user;
    private Show show;
    private List<ShowSeat> seats;
    private BookingStatus status;
}

interface SeatLockProvider {
    boolean lockSeats(Show show, List<Seat> seats, User user);
    void unlockSeats(Show show, List<Seat> seats);
    boolean isLocked(Show show, Seat seat);
}
```

Booking Flow

1. User selects city and movie.
2. System shows theaters and shows.
3. User selects seats.
4. System locks seats for a short time.
5. User pays.
6. If payment succeeds, booking is confirmed and seats become booked.
7. If payment fails or timeout happens, seats are unlocked.

Concurrency

Use one of:

- Database row lock on selected show seats.
- Optimistic locking with version number.
- Distributed lock with TTL.
- In-memory lock only for single-server systems.

Important APIs

```
List<Movie> searchMovies(String city);
List<Show> getShows(String movieId, String city);
List<ShowSeat> getAvailableSeats(String showId);
Booking createBooking(String userId, String showId, List<String> seatIds);
void confirmBooking(String bookingId, String paymentId);
void cancelBooking(String bookingId);
```

Edge Cases

- Seat already booked.
- Seat locked by another user.
- Payment succeeds after lock expiry.
- Payment fails.
- User refreshes after locking seats.
- Show is cancelled.
- Partial booking should not happen.

Follow-ups

- How do you price seats dynamically?
- How do you support coupons?
- How do you handle food ordering?
- How do you scale across cities?
- How do you support seat map rendering?

6. Chess

Problem

Design a chess game.

Requirements

Functional:

- Two players play on an 8x8 board.
- Support all pieces and legal moves.
- Track turns.
- Detect check, checkmate, stalemate.
- Support castling, en passant, pawn promotion.
- Maintain move history.

Non-functional:

- Rules should be extensible and testable.
- Move validation should be separate from UI.

Main Entities

- Game
- Board
- Cell
- Piece
- Move
- Player
- MoveValidator
- GameStatus

Class Design

```
enum Color {
    WHITE, BLACK
}

enum GameStatus {
    ACTIVE, WHITE_WON, BLACK_WON, DRAW, STALEMATE
}

class Cell {
    private int row;
    private int col;
    private Piece piece;
}

abstract class Piece {
    private Color color;
    private boolean killed;

    abstract boolean canMove(Board board, Cell from, Cell to);
}
```

```
class King extends Piece { }
class Queen extends Piece { }
class Rook extends Piece { }
class Bishop extends Piece { }
class Knight extends Piece { }
class Pawn extends Piece { }

class Move {
    private Player player;
    private Cell from;
    private Cell to;
    private Piece movedPiece;
    private Piece capturedPiece;
}

class Game {
    private Board board;
    private Player white;
    private Player black;
    private Player currentTurn;
    private List<Move> moveHistory;
    private GameStatus status;

    boolean makeMove(Player player, Cell from, Cell to) { }
}
```

Piece Movement

- King: one step any direction.
- Queen: horizontal, vertical, diagonal.
- Rook: horizontal, vertical.
- Bishop: diagonal.
- Knight: L shape, can jump.
- Pawn:
 - one step forward
 - two steps from initial position
 - diagonal capture
 - promotion on last rank
 - en passant special capture

Validation Layers

1. Source has a piece.
2. Piece belongs to current player.
3. Destination is inside board.
4. Destination does not contain same-color piece.
5. Piece movement rule is valid.
6. Path is clear if needed.
7. Move does not leave own king in check.

Edge Cases

- King moves into check.
- Castling through check.
- Pawn promotion choice.
- En passant only immediately after opponent pawn moves two squares.
- Stalemate with no legal moves but king not in check.
- Insufficient material draw.
- Threefold repetition, if included.

Follow-ups

- How do you implement undo?
 - How do you support online multiplayer?
 - How do you detect check efficiently?
 - How do you support chess variants?
 - How do you add a bot?
-

7. Snake and Ladder

Problem

Design a Snake and Ladder game.

Requirements

Functional:

- Board has cells from 1 to N.
- Snakes move player down.
- Ladders move player up.
- Multiple players take turns.
- Dice determines movement.
- First player to reach final cell wins.

Non-functional:

- Configurable board size.
- Configurable dice.
- Easy to add special cells.

Main Entities

- Game
- Board
- Cell
- Jump
- Snake
- Ladder

- Player
- Dice

Class Design

```
class Player {
    private String id;
    private String name;
    private int position;
}

class Jump {
    private int start;
    private int end;
}

class Board {
    private int size;
    private Map<Integer, Jump> jumps;

    int getFinalPosition(int position) { }
}

class Dice {
    private int count;
    private int min;
    private int max;

    int roll() { }
}

class SnakeAndLadderGame {
    private Board board;
    private Queue<Player> players;
    private Dice dice;
    private boolean finished;

    void start() { }
    void playTurn() { }
}
```

Game Flow

1. Current player rolls dice.
2. Calculate tentative position.
3. If position exceeds board size, player stays.
4. If position has snake or ladder, move to final position.
5. If final cell reached, player wins.
6. Otherwise, move player to back of queue.

Edge Cases

- Dice roll exceeds final cell.
- Snake at same cell as ladder.
- Snake or ladder creates cycle.
- Player lands on snake tail or ladder top.
- Multiple players at same cell.
- Need exact roll to win or not.

Follow-ups

- How do you support multiple dice?
 - How do you support custom board configuration?
 - How do you prevent invalid snakes/ladders?
 - How do you support special cells like skip turn?
 - How do you simulate the game?
-

8. Rate Limiter

Problem

Design a rate limiter that restricts how many requests a user/client can make in a time window.

Requirements

Functional:

- Allow or reject requests based on configured limits.
- Support per-user, per-IP, or per-API limits.
- Support multiple algorithms.
- Return retry-after information.

Non-functional:

- Very fast decision making.
- Thread-safe.
- Memory efficient.
- Distributed support may be needed.

Main Algorithms

Fixed Window:

- Count requests in a fixed time window.
- Simple but allows boundary bursts.

Sliding Window Log:

- Store timestamps of all requests.
- Accurate but memory-heavy.

Sliding Window Counter:

- Uses current and previous window weighted count.
- Good trade-off.

Token Bucket:

- Bucket refills at fixed rate.
- Request consumes one or more tokens.
- Allows controlled bursts.

Leaky Bucket:

- Requests are processed at constant rate.
- Smooths traffic.

Class Design

```
class RateLimitConfig {
    private int maxRequests;
    private long windowMillis;
}

class RateLimitResult {
    private boolean allowed;
    private long retryAfterMillis;
}

interface RateLimiter {
    RateLimitResult allow(String key);
}

class FixedWindowRateLimiter implements RateLimiter {
    private Map<String, Window> windows;
    private RateLimitConfig config;

    public synchronized RateLimitResult allow(String key) { }
}

class TokenBucketRateLimiter implements RateLimiter {
    private Map<String, TokenBucket> buckets;

    public synchronized RateLimitResult allow(String key) { }
}

class TokenBucket {
    private long capacity;
    private double tokens;
    private double refillRatePerMillis;
    private long lastRefillTimestamp;
}
```

Token Bucket Flow

1. Get bucket for key.
2. Refill tokens based on elapsed time.
3. If tokens $\geq$ cost, consume tokens and allow.
4. Otherwise reject and calculate retry time.

Distributed Design

Options:

- Redis **INCR** with expiry for fixed window.
- Redis sorted set for sliding log.
- Lua script for atomic token bucket updates.

Edge Cases

- Clock skew across servers.
- Memory cleanup for inactive keys.
- Huge number of unique users.
- Requests with different costs.
- Burst traffic.
- Redis failure.

Follow-ups

- How do you rate limit per API and per user together?
- How do you support different plans like free/pro/enterprise?
- How do you expose rate-limit headers?
- How do you handle distributed consistency?
- How do you test time-based logic?

9. Logger

Problem

Design a logging framework.

Requirements

Functional:

- Support log levels: DEBUG, INFO, WARN, ERROR, FATAL.
- Support multiple appenders: console, file, database, remote.
- Support configurable formatting.
- Support filtering by level.
- Support sync or async logging.

Non-functional:

- Logging should be fast.
- Logging should not crash the application.
- Thread-safe.
- Easy to add new appenders and formatters.

Main Entities

- `Logger`
- `LogManager`
- `LogEvent`
- `Appender`
- `Formatter`
- `LogLevel`
- `LogConfig`

Class Design

```
enum LogLevel {
    DEBUG, INFO, WARN, ERROR, FATAL
}

class LogEvent {
    private LogLevel level;
    private String message;
    private long timestamp;
    private String threadName;
    private Throwable throwable;
}

interface Appender {
    void append(LogEvent event);
}

interface Formatter {
    String format(LogEvent event);
}

class ConsoleAppender implements Appender { }
class FileAppender implements Appender { }

class Logger {
    private String name;
    private LogLevel level;
    private List<Appender> appenders;

    void debug(String message) { }
    void info(String message) { }
    void error(String message, Throwable throwable) { }
}

class LogManager {
```

```
private static Map<String, Logger> loggers;

static Logger getLogger(String name) { }

}
```

Design Patterns

- Singleton for `LogManager`.
- Factory for logger creation.
- Strategy for formatting.
- Chain of responsibility for filters.
- Observer-like fan-out to multiple appenders.

Async Logging

Use a blocking queue:

```
Application thread -> enqueue LogEvent -> background worker -> appenders
```

Important choices:

- What happens when queue is full?
 - block caller
 - drop logs
 - drop debug/info only
 - write synchronously

Edge Cases

- File appender cannot write.
- Disk is full.
- Appender throws exception.
- Log message is null.
- Logger config changes at runtime.
- Queue is full in async mode.
- Multiple threads log at once.

Follow-ups

- How do you implement log rotation?
- How do you support structured JSON logs?
- How do you support MDC/request IDs?
- How do you reload config dynamically?
- How do you avoid logging sensitive data?

10. File System

Problem

Design an in-memory file system.

Requirements

Functional:

- Create files and directories.
- Delete files and directories.
- Read and write file content.
- List directory contents.
- Support absolute paths.
- Support metadata like created time, modified time, size.

Non-functional:

- Path parsing should be reliable.
- Easy to add permissions.
- Directory operations should be efficient.

Main Entities

- `FileSystem`
- `FileSystemNode`
- `File`
- `Directory`
- `Path`
- `Permission`

Class Design

```
abstract class FileSystemNode {
    protected String name;
    protected Directory parent;
    protected long createdAt;
    protected long modifiedAt;

    abstract boolean isDirectory();
    abstract int size();
}

class File extends FileSystemNode {
    private StringBuilder content;

    void write(String data, boolean append) { }
    String read() { }
    int size() { }
}

class Directory extends FileSystemNode {
```

```

    private Map<String, FileSystemNode> children;

    void add(FileSystemNode node) { }
    void remove(String name) { }
    FileSystemNode getChild(String name) { }
    List<String> list() { }
}

class FileSystem {
    private Directory root;

    void mkdir(String path) { }
    void createFile(String path) { }
    void writeFile(String path, String content) { }
    String readFile(String path) { }
    List<String> ls(String path) { }
    void delete(String path) { }
}

```

Path Handling

For path `/a/b/c.txt`:

- Split by `/`.
- Start from root.
- Traverse `a`, then `b`.
- Last part is target file or directory.

Handle:

- root path `/`
- duplicate slashes
- trailing slash
- missing parent directory
- `.` and `..` if supported

Design Patterns

- Composite pattern:
 - file and directory both inherit from `FileSystemNode`.
- Command pattern can be used for undoable operations.

Edge Cases

- Create file where directory already exists.
- Create directory where file already exists.
- Delete non-empty directory.
- Read directory as file.
- Write to directory.
- Path does not exist.

- Invalid path.
- Root deletion.

Follow-ups

- How do you add permissions?
 - How do you support symbolic links?
 - How do you support file locking?
 - How do you persist this to disk?
 - How do you support search?
-

11. ATM

Problem

Design an ATM machine.

Requirements

Functional:

- Insert card.
- Authenticate using PIN.
- Check balance.
- Withdraw cash.
- Deposit cash/check.
- Transfer money.
- Print receipt.
- Eject card.

Non-functional:

- Secure authentication.
- Transaction consistency.
- ATM should handle hardware failures.
- Easy to add new transaction types.

Main Entities

- ATM
- Card
- Account
- BankService
- Transaction
- WithdrawalTransaction
- DepositTransaction
- TransferTransaction
- CashDispenser
- CashInventory

- ReceiptPrinter
- ATMState

State Pattern

ATM behavior changes by state:

- IdleState
- CardInsertedState
- AuthenticatedState
- TransactionSelectedState
- OutOfCashState
- MaintenanceState

Class Design

```
enum TransactionStatus {
    PENDING, SUCCESS, FAILED, CANCELLED
}

class Card {
    private String cardNumber;
    private String bankCode;
}

class Account {
    private String accountNumber;
    private Money balance;
}

abstract class Transaction {
    protected String id;
    protected Account account;
    protected Money amount;
    protected TransactionStatus status;

    abstract void execute();
}

class WithdrawalTransaction extends Transaction {
    private CashDispenser cashDispenser;
    private BankService bankService;

    void execute() { }
}

interface ATMState {
    void insertCard(Card card);
    void enterPin(String pin);
    void selectTransaction(TransactionType type);
    void cancel();
}
```

```
class ATM {  
    private ATMState state;  
    private Card currentCard;  
    private Account currentAccount;  
    private CashDispenser cashDispenser;  
    private BankService bankService;  
}
```

Withdrawal Flow

1. User inserts card.
2. ATM reads card.
3. User enters PIN.
4. Bank validates PIN.
5. User selects withdrawal.
6. ATM checks account balance.
7. ATM checks cash inventory.
8. Bank debits account.
9. ATM dispenses cash.
10. Receipt printed.
11. Card ejected.

Critical Consistency Concern

Withdrawal has two systems:

- bank account ledger
- physical cash dispenser

Possible failure:

- Account debited but cash not dispensed.

Handling:

- record transaction state.
- make debit and dispense part of a controlled workflow.
- if dispense fails, reverse the debit or mark transaction for reconciliation.
- maintain audit logs.

Cash Dispensing

Use denominations:

```
amount = 2800  
notes: 2000, 500, 100  
dispense: 1x2000, 1x500, 3x100
```

Greedy works for standard denominations, but for arbitrary denominations dynamic programming may be needed.

Edge Cases

- Invalid card.
- Wrong PIN multiple times.
- Card blocked.
- ATM out of cash.
- Requested amount unavailable by denominations.
- Insufficient account balance.
- Network failure with bank.
- Cash dispenser jam.
- User cancels transaction.
- Card not taken back.

Follow-ups

- How do you handle deposits?
 - How do you reconcile failed withdrawals?
 - How do you support multiple banks?
 - How do you add fraud detection?
 - How do you secure PIN entry?
-

Reusable Patterns Across These Problems

Strategy Pattern

Use when behavior can vary:

- Parking lot pricing.
- Parking spot assignment.
- Splitwise split strategy.
- Elevator scheduling.
- Rate limiter algorithm.
- Logger formatting.

State Pattern

Use when object behavior depends strongly on current state:

- ATM states.
- Elevator states.
- Booking lifecycle.
- Chess game state.

Factory Pattern

Use when object creation depends on type:

- Vehicle factory.
- Split strategy factory.
- Piece factory.
- Transaction factory.
- Appender factory.

Observer Pattern

Use for notifications:

- Booking confirmation.
- Splitwise expense notification.
- Library due-date reminder.
- Parking availability display.

Composite Pattern

Use for tree-like structures:

- File system.
- Organization hierarchy.
- Menu system.

Common Interview Checklist

For every design, prepare these:

- What are the main classes?
- What does each class own?
- Which class coordinates the main workflow?
- Which fields are immutable?
- Which methods mutate state?
- What validations are required?
- What happens on invalid input?
- What can run concurrently?
- Where can race conditions happen?
- Which parts should be interfaces?
- What design patterns are actually useful?
- How would you test the core logic?

Practice Prompts

For each problem, try answering:

1. Draw the class diagram.
2. Explain one end-to-end flow.
3. Write method signatures for the main service.
4. Identify two race conditions.

5. Explain how to add one new feature.
6. Mention one thing you intentionally did not design deeply and why.

Good LLD answers are not about memorizing classes. They are about making responsibilities obvious and showing that the design can survive small changes without becoming messy.